Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

SISTEMA DE AUTORIZACIÓN Y REPORTES DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS MEDIANTE API EN C#

Carné: 202500268 – JOSÉ MARCOS VÁSQUEZ ROMÁN

Resumen

El presente ensayo expone el diseño y desarrollo de un sistema de autorización de documentos tributarios electrónicos (DTE) orientado a simular servicios de la administración tributaria guatemalteca. La solución integra una API HTTP en C#, una interfaz web con Razor Pages y almacenamiento persistente en archivos JSON. Su aporte técnico consiste en aplicar programación orientada a objetos, tipos de dato abstracto propios, expresiones regulares y reglas de verificación para controlar la calidad de cada solicitud. El sistema administra el Registro Tributario Unificado, valida NIT de emisores y receptores, verifica importes e IVA, rechaza referencias duplicadas y genera autorizaciones con correlativos diarios. También consolida facturas aprobadas, rechazos, movimientos de IVA y montos por fecha, presentados mediante consultas, gráficas y documentos PDF. Se examinan las decisiones de arquitectura, los mecanismos de persistencia, las pruebas y sus limitaciones. Se concluye que separar responsabilidades facilita el mantenimiento y la trazabilidad, aunque una puesta en producción exigiría reforzar seguridad, concurrencia distribuida y gestión transaccional del almacenamiento.

Palabras clave

API HTTP, DTE, C#, programación orientada a objetos, validación tributaria.

Abstract

This essay presents the design and development of an electronic tax document (DTE) authorization system intended to simulate services provided by the Guatemalan tax administration. The solution integrates an HTTP API written in C#, a Razor Pages web interface, and persistent storage in JSON files. Its main technical contribution is the application of object-oriented programming, custom abstract data types, regular expressions, and verification rules to control incoming requests. The system manages the taxpayer registry, checks issuer and receiver tax identifiers, validates amounts and value-added tax, rejects duplicate references, and generates daily authorization sequences. It also summarizes approved invoices, rejected requests, tax movements, and invoiced amounts by date through queries, charts, and PDF documents. The essay discusses architectural decisions, persistence mechanisms, test scenarios, and limitations. It concludes that separating responsibilities improves maintainability and traceability, although production deployment would require stronger security, distributed concurrency controls, and transactional storage management.

Keywords

HTTP API, electronic tax document, C#, object-oriented programming, tax validation.

Introducción

La autorización de documentos tributarios electrónicos requiere verificar contribuyentes, importes y datos formales antes de emitir un identificador único. Este trabajo presenta una solución académica para ese problema en la que una interfaz web consulta un servicio HTTP y este administra las reglas de negocio y la persistencia. El desarrollo responde a los requisitos del Proyecto 3 de Introducción a la Programación y Computación 2 (Escuela de Ciencias y Sistemas, 2026). Su objetivo es demostrar que el uso disciplinado de clases, validadores y estructuras propias permite construir un sistema verificable y ampliable. El análisis aborda la arquitectura, la validación de facturas, los mecanismos de almacenamiento, los reportes tributarios y la evaluación mediante casos concretos. Asimismo, distingue los resultados demostrables de un prototipo académico de las garantías adicionales que necesitaría una plataforma tributaria real.

Desarrollo del tema

1. Arquitectura y distribución de responsabilidades

La solución se organiza en dos programas y una biblioteca de dominio. SatDte.Web corresponde al simulador web desarrollado con Razor Pages; desde sus formularios se cargan archivos JSON, se seleccionan fechas y contribuyentes y se visualizan los resultados. SatDte.Api publica rutas HTTP independientes que admiten peticiones de otros clientes, como Postman. Finalmente, SatDte.Core concentra modelos, estructuras, validadores, servicios, acceso a archivos y exportación de reportes. Esta separación evita que las reglas tributarias dependan de una página específica y permite verificar la lógica de forma aislada.

En ASP.NET Core, las acciones de un controlador pueden gestionar solicitudes HTTP y devolver respuestas estructuradas, mientras Razor Pages organiza la experiencia en páginas y modelos de interacción (Microsoft, s. f.-a; Microsoft, s. f.-b). El proyecto aprovecha esta complementariedad: el frontend no decide si una factura es válida, sino que presenta la respuesta emitida por la API. De manera equivalente, la API no necesita conocer el diseño visual para producir consultas o archivos PDF. La arquitectura resultante hace explícito dónde se recibe información, dónde se procesa y dónde se conserva.

Tabla I.

Componentes y responsabilidades principales.

COMPONENTE	RESPONSABILIDAD
SatDte.Web	Carga de JSON, formularios, gráficas y descarga de PDF.
SatDte.Api	Rutas HTTP y respuestas JSON para RTU, DTE y reportes.
SatDte.Core	Modelos, reglas, TDA, persistencia y exportador PDF.
SatDte.Verificacion	Comprobaciones de validación, continuidad y consultas.

Fuente: elaboración propia, 2026.

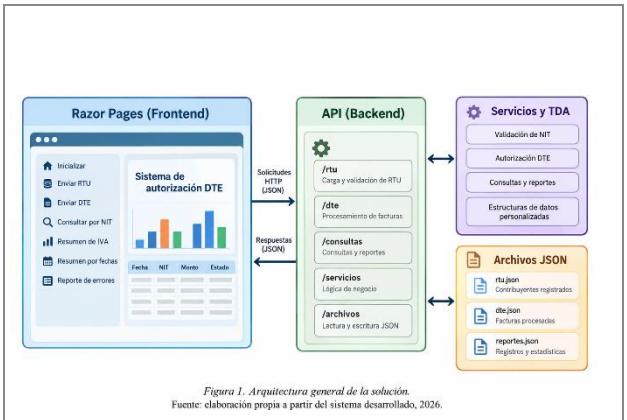


Figura 1. Arquitectura general de la solución.
Fuente: elaboración propia a partir del sistema desarrollado, 2026.

Pages → API → servicios, TDA y archivos JSON.

Figura 1. Arquitectura general de la solución.

Fuente: elaboración propia a partir del sistema desarrollado, 2026.

2. Estructuras de datos propias y almacenamiento

La programación orientada a objetos se expresa mediante clases que representan responsabilidades identificables. Contribuyente y RegistroFactura modelan datos; ValidadorNit y ValidadorFactura comprueban reglas; ServicioRtu, ServicioFacturas y ServicioReportes coordinan procesos. El diseño evita centralizar todas las operaciones en una clase monolítica. Además, las dependencias de los servicios se suministran desde la configuración del backend, lo que hace más sencilla su sustitución para realizar pruebas.

Para almacenar información en memoria se implementaron dos tipos de dato abstracto propios. TablaRtu administra contribuyentes mediante un arreglo que amplía su capacidad cuando es necesario, y ofrece búsqueda, inserción, actualización y limpieza. RegistroSolicitudes administra registros de facturas en otro arreglo dinámico, con funciones para comprobar referencias previas y recuperar el último correlativo de una fecha. Su enfoque es comprensible y suficiente para los archivos de prueba; sin embargo, la búsqueda lineal puede convertirse en un costo relevante cuando crezca el número de registros. En ningún caso estas estructuras requieren Node<T> ni una lista enlazada.

La persistencia se realiza mediante ArchivoRtu y ArchivoFacturas, que serializan y recuperan datos en archivos. El contenido almacenado permite mantener contribuyentes, aprobaciones y rechazos después de reiniciar el servidor. Como formato de intercambio, JSON facilita representar arreglos de objetos con campos identificables y tipos básicos; su estructura general se encuentra estandarizada en RFC 8259 (Bray, 2017). El uso de archivos resuelve la persistencia exigida en el ejercicio, aunque no equivale a una base de datos transaccional para operación distribuida.

3. Validación tributaria y autorización de DTE

Un lote de facturas se recibe sin alterar los nombres de campos acordados: TIEMPO, REFERENCIA, NIT_EMISOR, NIT_RECEPTOR, VALOR, IVA y TOTAL. El servicio extrae de TIEMPO la fecha y hora mediante una expresión regular, ignorando texto incidental como el lugar de emisión. Después valida referencias de hasta 40 caracteres, importes positivos y coincidencia entre las sumas. Esta separación entre el mensaje original y los campos útiles responde a la instrucción de no rechazar información adicional que no sea relevante para la autorización (Escuela de Ciencias y Sistemas, 2026).

El NIT se comprueba aplicando el algoritmo de dígito verificador del enunciado. El procedimiento recorre los dígitos base, calcula una suma ponderada y transforma el módulo 11 en el carácter final

esperado; cuando el resultado es diez, el verificador es K. Se comprueba tanto la forma como la existencia del contribuyente en el RTU. Para el IVA se calcula el 12 % del valor y se redondea a dos decimales; el TOTAL debe coincidir exactamente con VALOR más IVA. Los cálculos monetarios utilizan decimal, apropiado para evitar errores comunes de representación binaria en cantidades monetarias.

Tabla II.

Reglas principales aplicadas a cada solicitud.

CAMPO	REGLA DE VALIDACIÓN
TIEMPO	Fecha dd/mm/aaaa y hora de 00:00 a 23:59.
REFERENCIA	Obligatoria, hasta 40 caracteres y sin duplicados.
NIT	Dígitos base y verificador válido; existencia en RTU.
VALOR	Monto positivo con hasta dos decimales.
IVA	Redondear(VALOR × 0,12; 2).
TOTAL	Igual a VALOR + IVA.

Fuente: elaboración propia, 2026.

Solo una solicitud sin errores obtiene autorización. El código se construye concatenando la fecha local de Guatemala en formato aaaammdd y un número consecutivo de ocho posiciones, que inicia de nuevo cada día. La fecha que identifica al correlativo es la fecha de autorización, no necesariamente la fecha contenida en el documento. Los rechazos se conservan para auditoría y no consumen números aprobados; las referencias ya procesadas se detectan como repetidas. El servicio protege el proceso con un bloqueo dentro de la aplicación y prepara una copia de trabajo antes de guardar los datos, procurando coherencia entre el archivo y el estado en memoria.

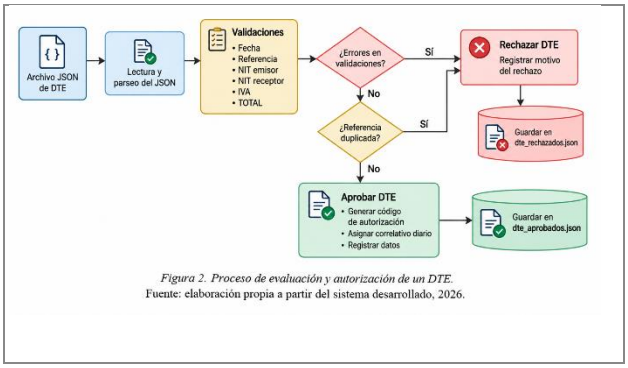


Figura 2. Proceso de evaluación y autorización de un DTE.
Fuente: elaboración propia a partir del sistema desarrollado, 2026.

4. Consultas, indicadores y presentación de resultados

Una vez almacenadas las solicitudes, la API puede recuperar resúmenes por contribuyente, IVA, rango de fechas y recepción de documentos. Las consultas financieras consideran únicamente facturas aprobadas, pues una solicitud rechazada no representa una autorización tributaria válida. En cambio, el informe de recepción incluye también intentos rechazados y sus motivos; esta diferencia evita mezclar los movimientos reconocidos con el análisis de la calidad de los mensajes recibidos.

El informe por contribuyente permite escoger facturas emitidas o recibidas para un NIT en un intervalo. El resumen de IVA muestra, por día, cuánto IVA figura como emitido y cuánto como recibido para cada contribuyente. El resumen por rango agrupa facturas aprobadas y ofrece importes con IVA o sin IVA. Finalmente, el informe de errores separa cantidades recibidas, autorizadas y rechazadas, además de los distintos motivos de invalidación. Para las agrupaciones se utiliza la fecha del documento; cuando una fecha de entrada es inválida, el informe de recepción usa la fecha de llegada a Guatemala.

Tabla III.

Consultas disponibles en la API.

CONSULTA	RESULTADO PRINCIPAL
Contribuyente	Cantidad de facturas e importe por fecha y NIT.

IVA por fecha	IVA emitido, recibido y diferencia por NIT.
Rango de fechas	Monto autorizado con IVA o sin IVA.
Recepciones	Solicitudes, autorizaciones y errores por día.

Fuente: elaboración propia, 2026.

La interfaz emplea gráficas SVG para comparar IVA por NIT y montos autorizados por fecha, sin depender de servicios de visualización externos. Para facilitar la conservación de resultados, se implementó un generador de archivos PDF que funciona en C# y puede ser invocado directamente desde los endpoints de la API. La descarga no depende de una captura del navegador: la misma consulta que alimenta el reporte produce datos para una salida imprimible. Este mecanismo muestra una ventaja de separar la presentación del cálculo tributario.

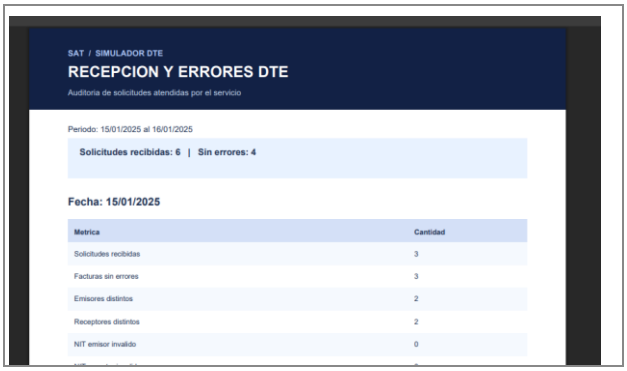


Figura 3. Consulta tributaria con gráfica y opción de PDF.
Fuente: elaboración propia a partir del sistema desarrollado, 2026.

5. Pruebas funcionales y evaluación de resultados

La verificación del prototipo se planteó en escenarios acumulativos, desde la carga del RTU hasta los reportes consolidados. Un archivo inicial incorpora dos NIT válidos y descarta un NIT con verificador incorrecto; su resultado previsto es dos contribuyentes nuevos y un NIT inválido. Después se remiten lotes DTE con facturas aceptables y con errores, para comprobar que los rechazos queden identificados sin detener el procesamiento completo. Se revisa también que el correlativo no vuelva a utilizarse después de reiniciar las aplicaciones, y que un nuevo día comience con el consecutivo uno.

En el conjunto de datos de consulta distribuido con el proyecto, seis solicitudes permiten comprobar cuatro autorizaciones y dos rechazos. Para los documentos autorizados, el total del día 15/01/2025 asciende a 392,00 y el del 16/01/2025 a 84,00, lo que produce 476,00 con IVA. Si se solicita el valor sin IVA, los importes son 350,00 y 75,00, respectivamente, con una suma de 425,00. Estos números constituyen criterios de aceptación reproducibles: si una modificación altera el resultado, el problema puede localizarse en la lectura, los filtros, los cálculos o la visualización.

Tabla IV.
Escenarios de comprobación previstos.

PRUEBA	RESULTADO ESPERADO
Importación RTU	2 nuevos y 1 NIT inválido.
Lote de reportes	4 aprobadas y 2 rechazadas.
Rango con IVA	392 + 84 = 476.
Rango sin IVA	350 + 75 = 425.
Persistencia	Datos conservados tras reiniciar.

Fuente: elaboración propia, 2026.

La aplicación incorpora también un proyecto de verificación que ejercita reglas, estructura de los datos y persistencia. Las pruebas funcionales manuales complementan esa verificación con acciones reales desde Visual Studio y el navegador. Aun con resultados correctos en los ejemplos, la calidad de una solución debe evaluarse por su comportamiento ante entradas extremas: lotes vacíos, fechas imposibles, duplicados repetidos y archivos dañados. Dichos casos aportan más evidencia que una sola captura de funcionamiento satisfactorio.

6. Alcances, limitaciones y mejoras posibles

El diseño demuestra la operación de una API con una interfaz independiente, pero conserva limitaciones propias del alcance universitario. El almacenamiento en archivos obliga a considerar respaldos, escrituras interrumpidas, volumen de

datos y coordinación de múltiples procesos. El bloqueo implementado protege instancias dentro de un proceso del servicio; no reemplaza una secuencia centralizada o una transacción cuando existen varias réplicas del servidor. Asimismo, una aplicación expuesta públicamente necesitaría autenticación, autorización por roles, protección de información tributaria, control de peticiones y monitoreo permanente, aspectos que no se presentan como resueltos en este prototipo.

Una evolución razonable consistiría en incorporar almacenamiento transaccional, claves únicas para las referencias, índices para consultas frecuentes y pruebas automatizadas ejecutadas dentro del proceso de integración continua. También podría ampliarse la documentación de contratos HTTP y la supervisión de errores sin modificar los formatos de respuesta obligatorios. Estas mejoras no descalifican el enfoque adoptado: muestran por qué conviene separar servicios, modelos y persistencia desde las primeras versiones, ya que una sustitución posterior sería menos invasiva.

Conclusiones

El proyecto evidencia que los principios de programación orientada a objetos pueden aplicarse a un servicio HTTP que reproduce reglas tributarias concretas. La división entre Razor Pages, controladores y clases de dominio permitió conservar las mismas reglas cuando se añadieron reportes, gráficas y documentos PDF. Los TDA implementados con arreglos dinámicos demostraron que es posible administrar el RTU y las solicitudes sin depender de Node<T>, aunque con límites de eficiencia conocidos.

La validación previa de NIT, fechas, importes y referencias evita autorizar documentos inconsistentes. A la vez, registrar tanto aprobaciones como rechazos permite diferenciar el monto efectivamente autorizado de los errores de los mensajes recibidos. Los casos de prueba con resultados esperados ofrecen una base objetiva para revisar cambios y proteger el correlativo diario. Por último, el trabajo confirma que un prototipo académico funcional puede servir como punto de partida técnico, siempre que se reconozcan sus necesidades futuras de seguridad, escalabilidad y transacciones.

Referencias bibliográficas

Bray, T. (2017). The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Escuela de Ciencias y Sistemas. (2026). Proyecto 3, Introducción a la Programación y Computación 2 [Enunciado de laboratorio]. Universidad de San Carlos de Guatemala.

Microsoft. (s. f.-a). Create web APIs with ASP.NET Core. Microsoft Learn. https://learn.microsoft.com/aspnet/core/web-api/

Microsoft. (s. f.-b). Introduction to Razor Pages in ASP.NET Core. Microsoft Learn. https://learn.microsoft.com/aspnet/core/razor-pages/

Anexo A. Diagrama de clases (espacio reservado)

Sustituir el recuadro siguiente por el diagrama de clases definitivo de SatDte.Core, SatDte.Api y SatDte.Web. Es un anexo independiente del contenido principal del ensayo.

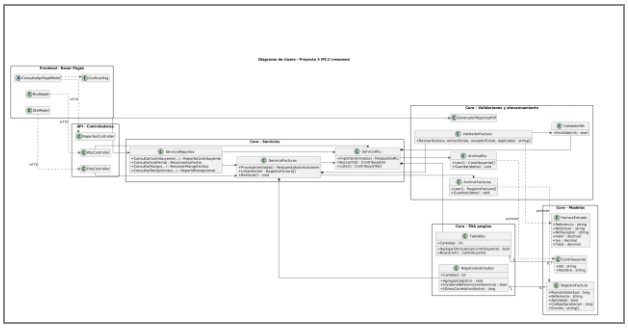


Figura A1. Diagrama de clases de la solución.
Fuente: elaboración propia a partir del sistema desarrollado, 2026.